



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

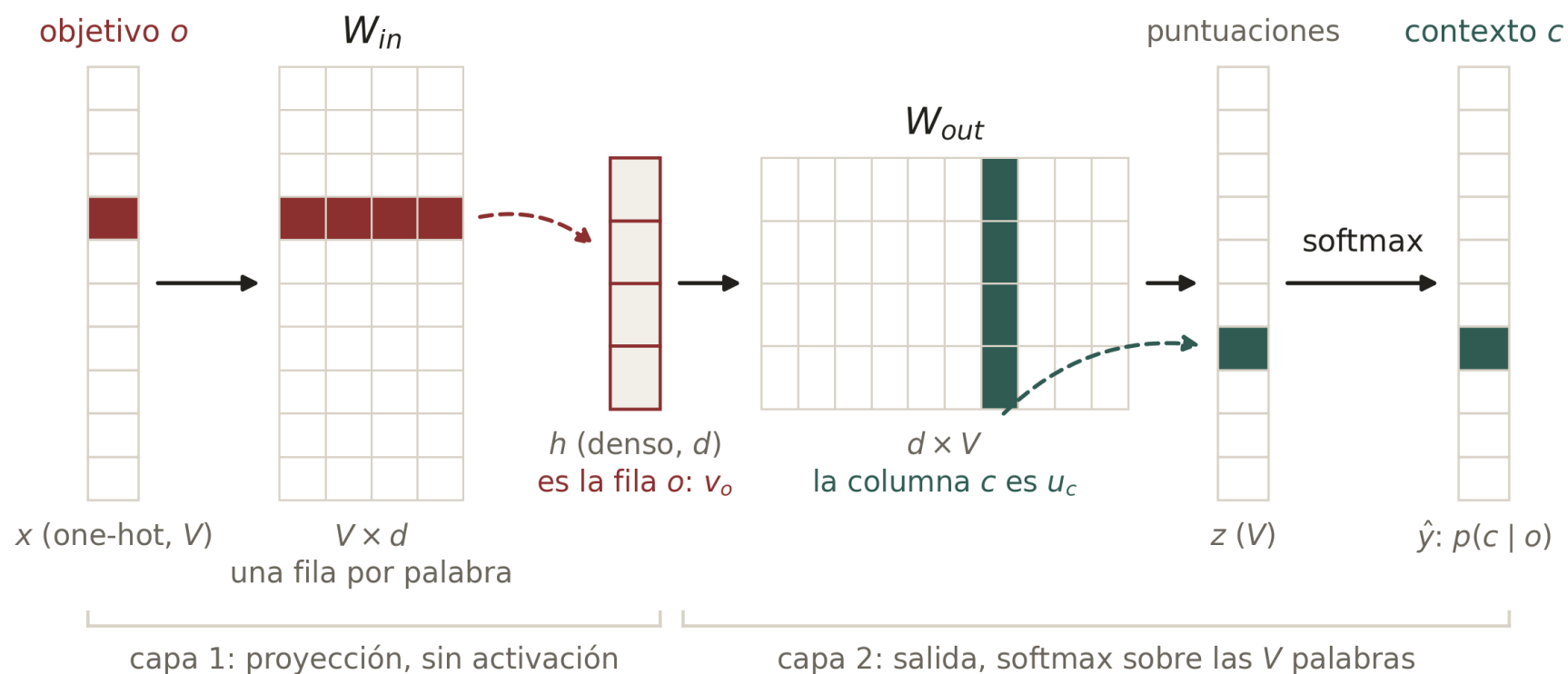
Procesamiento de Lenguaje Natural

github.com/marcelomendoza/IIC3670

Marcelo Mendoza

La red de word2vec: dos capas

Una palabra entra en one-hot y salen V probabilidades. Las matrices de pesos son las dos capas.



Las ecuaciones de transferencia

Tres pasos, uno por flecha de la figura: seleccionar, puntuar, normalizar.

$$h = W_{in}^T x = v_o$$

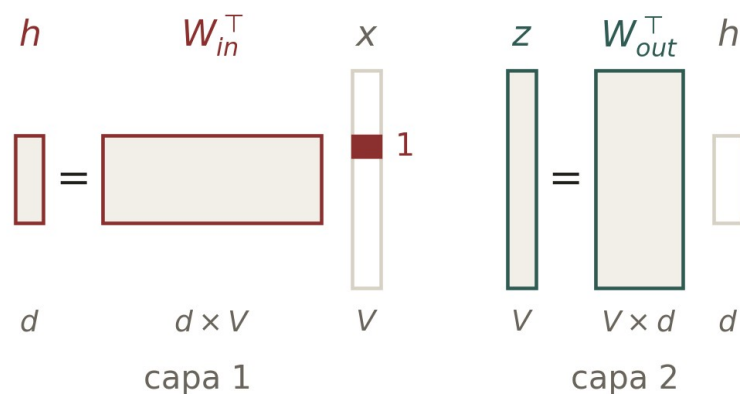
elige la fila o

$$z = W_{out}^T h, \quad z_c = u_c^T v_o$$

una puntuación por palabra

$$\hat{y} = \text{softmax}(z) : \quad p(c | o) = \frac{e^{u_c^T v_o}}{\sum_{k=1}^V e^{u_k^T v_o}}$$

de puntuaciones a probabilidades



Seleccionar

La primera capa no calcula: el one-hot elige una fila de la matriz de entrada, y esa fila es el vector del objetivo.

Puntuar

La segunda da una puntuación por palabra: el producto punto entre la fila del objetivo y la columna del candidato.

Normalizar

La softmax lleva esas V puntuaciones a una distribución, que es lo que se compara contra el contexto observado.

Bolsa de palabras y embedding no son lo mismo

Uno describe un documento entero y el otro una sola palabra. No son dos versiones de la misma idea.

$\mathbf{x} \in \mathbb{R}^V$: el documento

una coordenada por palabra

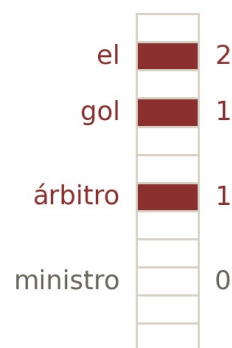
$\mathbf{v}_w \in \mathbb{R}^d$: la palabra

ninguna coordenada tiene nombre

$V \approx 30.000 \gg d \approx 300$

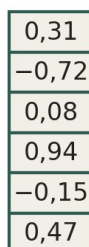
cien veces más corto, y sin ceros

bolsa de palabras



$\mathbf{x}$: un documento
 $V \approx 30.000$, casi todo ceros

word embedding



$\mathbf{v}_w$: una palabra
 $d \approx 300$, nada en cero

Describen cosas distintas

La bolsa de palabras es un vector por documento; el embedding, un vector por palabra. Es la diferencia que más se olvida.

Disperso y denso

El BOW tiene una coordenada por palabra y casi todas valen cero. El embedding tiene unos cientos, ninguna en cero.

Contado y aprendido

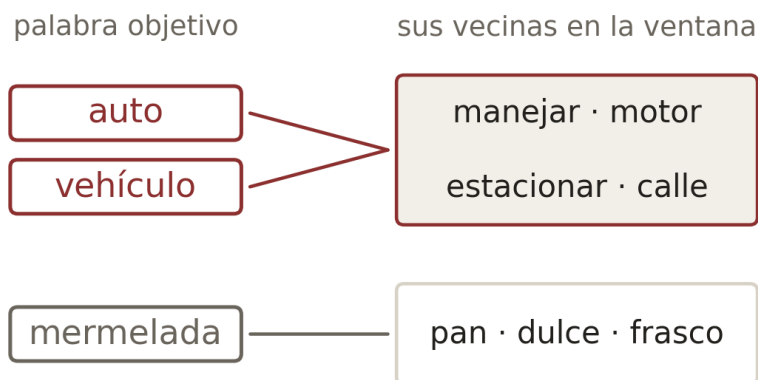
El BOW se cuenta y cada coordenada se puede nombrar. El embedding se entrena y ninguna coordenada dice nada.

Palabras relacionadas quedan cerca en ángulo

La relación no se declara en ninguna parte: sale de que dos palabras aparecen rodeadas de las mismas.

$$\cos \theta = \frac{u^\top v}{\|u\| \|v\|} \quad \theta \text{ pequeño} \Rightarrow \cos \theta \approx 1 \quad \theta \approx 90^\circ \Rightarrow \cos \theta \approx 0$$

Por eso una consulta que no comparte ninguna palabra con el documento da cero en tf-idf y en BM25, y en el espacio de embeddings sigue estando cerca.



«auto» y «vehículo» no aparecen juntas, pero la ventana les da casi las mismas vecinas.



El entrenamiento las alinea: ángulo pequeño entre las relacionadas, grande con las demás.

Se conectan a través de una vecina común

«no encontré dónde **estacionar** el _____»

las dos caben

(auto, **estacionar**) y (vehículo, **estacionar**) son pares

(auto, vehículo) no



Nunca son un par

Las dos palabras no aparecen juntas, así que el entrenamiento jamás ve ese par. La conexión es indirecta.

La vecina común las une

Las dos aparecen con «estacionar», y cada aparición empuja su vector hacia el mismo vector de contexto.

Bajar la pérdida es alinear

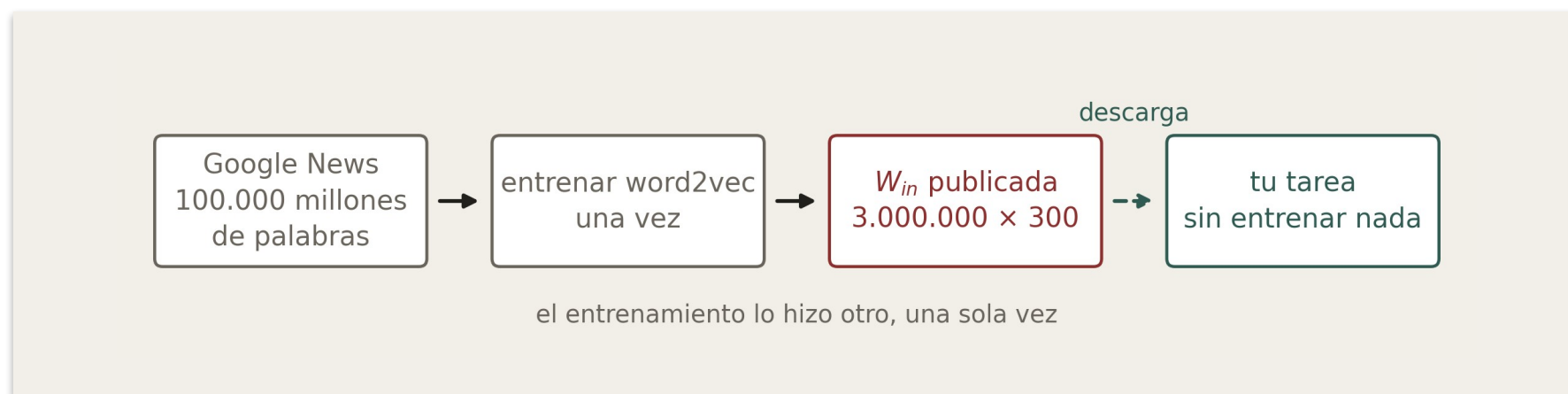
No hay otra manera de subir ese producto punto: la alineación es el objetivo, no un efecto lateral.

$$\mathcal{L} = -\frac{1}{|S|} \sum_{(o,c) \in S} \left[\mathbf{y} \log \sigma(u_c^\top v_o) + (1 - \mathbf{y}) \log (1 - \sigma(u_c^\top v_o)) \right]$$

con $y = 1$ el término que sobrevive baja cuando sube el producto punto: el objetivo y su vecina se alinean

Preentrenar: los pesos ya están hechos

Nadie entrena word2vec desde cero: se descarga la tabla de vectores de un modelo entrenado sobre un corpus que uno no tiene.



referee
coach
soccer
basketball
football
tennis
automobile
motorcycle
vehicle
car
sedan truck
vehículos
deportes
comida
computación
bread
dessert
cheese
pasta
pizza
restaurant
internet
computer
laptop
database
software
algorithm

Entrenar una vez, usar muchas

El costo del entrenamiento lo pagó otro. Bajar los vectores y usarlos cuesta minutos.

Lo que se descarga es la primera matriz

Tres millones de palabras y frases, vectores de largo 300. La segunda capa no viene, y no hace falta.

Los grupos se forman solos

Nadie le dijo al modelo qué es un auto ni qué es una pizza. La agrupación sale de los contextos.

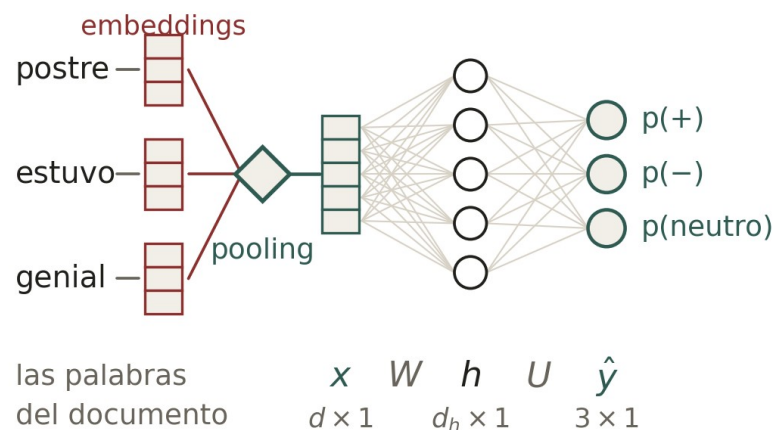
Pooling: de los vectores de palabra al del documento

Las palabras ya tienen vector. El documento se arma agregando los de sus palabras.

$$x = \text{pool}(e(w_1), \dots, e(w_n)) \quad \text{el documento en un vector de largo } d$$

$$\text{media} : x_j = \frac{1}{n} \sum_i e(w_i)_j \quad \text{máximo} : x_j = \max_i e(w_i)_j$$

$$h = \sigma(Wx + b), \quad z = Uh, \quad \hat{y} = \text{softmax}(z)$$



El pooling arma el documento

Los n vectores de las palabras se juntan en uno solo de largo d , y el documento vuelve a ser un vector.

La agregación se elige

La media es lo habitual; el máximo por coordenada y la suma también se usan. Es una decisión de diseño.

Encima va el clasificador de siempre

Una capa oculta y una softmax: lo mismo de la clase 2, con otra entrada.

Palabras que el modelo nunca vio

El vocabulario está congelado: lo que no está en la tabla no tiene vector. La salida es dejar de mirar la palabra entera.

$\text{correr} \longrightarrow \{ \text{<co, cor, orr, rre, rer, er>} \} \quad n=3, \text{ con marcas de borde}$

$\mathcal{G}_o \subset \{1, \dots, G\} \quad \text{los subwords del objetivo } o$

subwords de 3 caracteres, con marcas de borde
«correr»

<co cor orr rre rer er>

«corriendo»

<co cor orr rri rie ien end ndo do>

comparten 3 subwords: la raíz queda a la vista

Fuera del vocabulario, sin vector

Para una palabra nueva no hay vector. La tabla tiene las palabras que vio y nada más.

La palabra se parte en subwords

Se le ponen marcas de borde y se toman todos los subwords de n caracteres. La palabra entera cuenta como un subword más.

Los subwords se comparten

«correr» y «corriendo» comparten la raíz, así que comparten subwords.

Un vector por subword, y la palabra es la suma

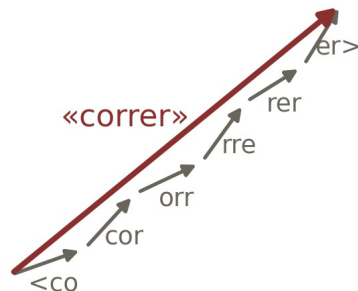
Si los vectores se entrenan a nivel de char n-grama, el de la palabra se arma agregando los de sus subwords.

$$v_g \in \mathbb{R}^d$$

un vector por subword, no por palabra

$$\mathbf{v}_o = \sum_{g \in \mathcal{G}_o} v_g$$

la palabra es la suma de sus subwords



el vector de la palabra es la suma de los de sus subwords

Los vectores son de los subwords

El modelo deja de guardar un vector por palabra y guarda uno por subword. Ese catálogo es chico y no crece.

La palabra se arma agregando

La suma es lo habitual, pero es el mismo pooling de la lámina anterior: la media también sirve.

Una palabra nueva ya tiene vector

Nunca vista, pero sus subwords sí. El vector sale igual, y queda cerca de las palabras que comparten subwords.

Composicionalidad: pocos símbolos, todas las palabras

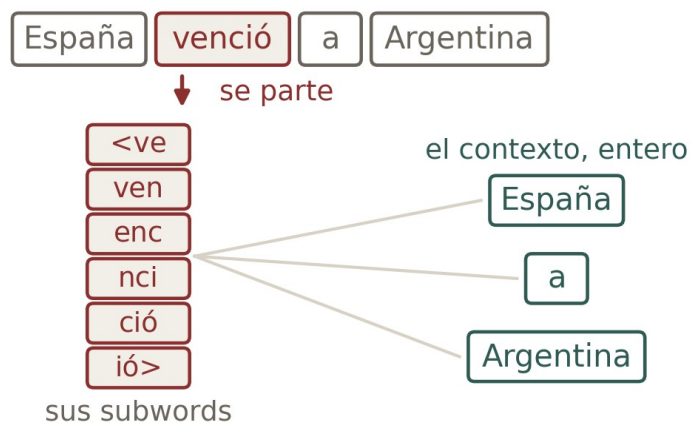
Los pares de entrenamiento son los mismos de la clase pasada. Lo que cambió es la unidad con que se escribe el objetivo.

$$\#\{\text{trigramas}\} = |\Sigma|^3 = 35^3 = \mathbf{42.875}$$

33 letras y 2 bordes

6.051 observados para 19.968 palabras distintas

el catálogo se llena



los pares son los mismos de la clase pasada:
lo único que cambió es cómo entra el objetivo

Los pares no cambian

El skip-grama sigue siendo un objetivo y una palabra de su ventana. El objetivo ahora entra por sus subwords.

El catálogo es finito

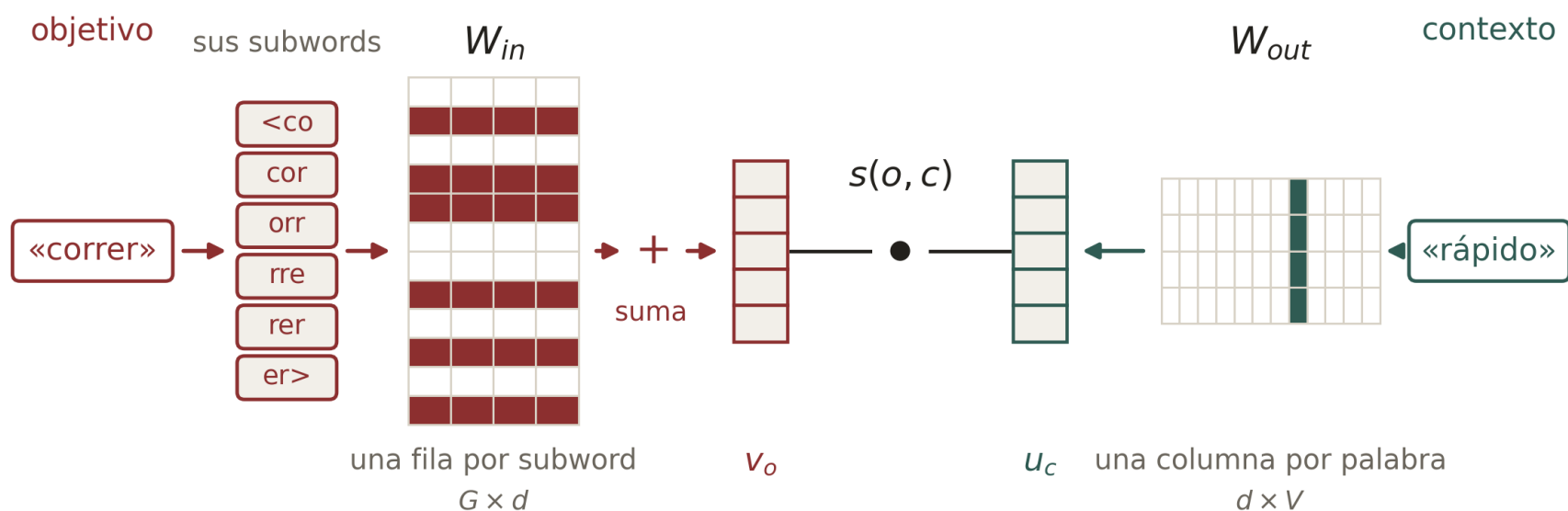
Con 35 símbolos hay 42.875 trigramas posibles. En un corpus de 20.000 palabras distintas aparecen 6.051.

Composicionalidad

Diez veces más texto multiplica el vocabulario por 5 y los trigramas por menos de 2. El alfabeto no crece.

El objetivo por subwords, el contexto entero

La primera capa cambió de unidad y la segunda no: es la misma red de antes, partida por la mitad.



Las ecuaciones de transferencia con subwords

$$\mathbf{x}_o = \sum_{g \in \mathcal{G}_o} e_g \in \{0, 1\}^G$$

un uno por subword, no uno solo

$$\mathbf{v}_o = W_{in}^\top \mathbf{x}_o = \sum_{g \in \mathcal{G}_o} v_g$$

selecciona filas y las suma

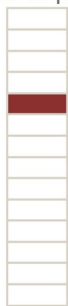
$$\mathbf{z} = W_{out}^\top \mathbf{v}_o, \quad \mathbf{z}_c = \mathbf{u}_c^\top \mathbf{v}_o$$

igual que antes

$$\hat{y} = \text{softmax}(\mathbf{z})$$

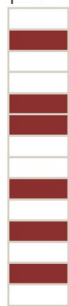
igual que antes

word2vec
un uno: la palabra



largo V

con subwords
un uno por subword



largo G

De one-hot a multi-hot

El objetivo ya no entra con un solo uno sino con uno por subword, sobre un vector de largo G en vez de V.

La capa 1 no cambió

Sigue siendo la matriz de entrada por el vector de entrada. Con varios unos, selecciona varias filas y las suma.

La capa 2 tampoco

Las dos últimas líneas son las mismas de la lámina 3, sin cambiarles una letra.

Por qué solo un lado se parte

La puntuación es la de siempre, con un lado armado por partes. La pérdida no cambia en nada.

$$s(o, c) = \sum_{g \in \mathcal{G}_o} u_c^\top v_g = u_c^\top \sum_{g \in \mathcal{G}_o} v_g = u_c^\top \mathbf{v}_o \quad \text{por linealidad}$$

$$\mathcal{L} = -\frac{1}{|S|} \sum_{(o, c) \in S} \left[\mathbf{y} \log \sigma(s(o, c)) + (1 - \mathbf{y}) \log (1 - \sigma(s(o, c))) \right] \quad \text{solo cambió } s$$

W_{in}

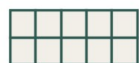


el objetivo

tiene que valer para cualquier palabra, vista o no

por eso se parte en subwords

W_{out}



el contexto

solo existe mientras se entrena, y después se descarta

Sumar primero o después da igual

El producto punto es lineal, así que sumar los subwords y multiplicar una vez es lo mismo que multiplicar cada uno y sumar.

La misma pérdida de antes

La del muestreo negativo, con la etiqueta diciendo si el par se observó o se sorteó. Lo único distinto es de dónde sale s .

Solo un lado necesita generalizar

El vector del objetivo es el que se guarda y tiene que valer para palabras nuevas. El del contexto se descarta.

fastText: los embeddings que se descargan

Bojanowski, Grave, Joulin y Mikolov. «Enriching Word Vectors with Subword Information». TACL 5 (2017), 135 a 146.

294 idiomas

Vectores skip-gram entrenados sobre Wikipedia, todos de largo 300, con el español entre ellos.

los del paper

157 idiomas

Versión posterior, entrenada sobre Common Crawl y Wikipedia, también de largo 300.

Grave y otros, LREC 2018

Nunca falta un vector

Una palabra que no está en la tabla igual recibe uno, armado con sus subwords.

se acabó el OOV

Diez minutos

Mil millones de tokens en CPU, según los autores. Entrenar los propios sobre un dominio es viable.

sin GPU

Vectores estáticos

«banco» tiene un solo vector, promedio de todos sus sentidos, y no cambia según la oración.

el límite de la unidad

Cuidado con el nombre

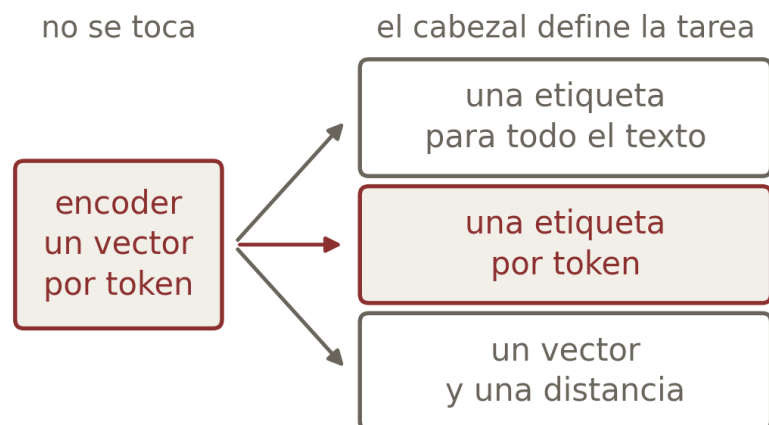
La misma biblioteca trae además un clasificador de texto, de otro paper. No es lo mismo que esto.

no confundir

Un encoder, muchos cabezales

Los vectores por token son la entrada de cualquier cosa. Lo que se ponga encima decide la tarea.

	Neruda	nació	en	Parral	en	1904
POS	PROPN	VERB	ADP	PROPN	ADP	NUM
NER	PER	O	O	LUG	O	FECHA



El encoder no cambia

Los mismos vectores sirven para clasificar, para etiquetar y para comparar. Lo que cambia es el cabezal.

Sequence labeling

Una etiqueta por token y no una por documento: la salida tiene el mismo largo que la entrada.

Las tareas

Entidades nombradas, categorías gramaticales, segmentación en sintagmas. Todas tienen la misma forma.

La matriz de transiciones

Una softmax por token decide cada uno por su lado. El CRF agrega una tabla que dice qué etiqueta puede seguir a cuál.

$$P \in \mathbb{R}^{n \times K}$$

lo que la red da: un puntaje por token y etiqueta

$$A \in \mathbb{R}^{K \times K}$$

lo nuevo: un puntaje por par de etiquetas seguidas

$$s(x, y) = \sum_{i=1}^n P_{i, y_i} + \sum_{i=1}^n A_{y_{i-1}, y_i}$$

el puntaje de la secuencia entera

		etiqueta siguiente				
		B-PER	I-PER	B-LUG	I-LUG	O
etiqueta anterior	B-PER	-8,37	-0,41	-7,27	nunca	-1,10
	I-PER	-6,48	-1,34	-5,13	nunca	-0,32
	B-LUG	-5,93	nunca	-5,24	-1,53	-0,25
	I-LUG	nunca	nunca	-6,85	-0,83	-0,57
	O	-4,08	nunca	-3,94	nunca	-0,04

medido sobre 8.323 frases de conll2002 en español

Una tabla de K por K

Una entrada por cada par ordenado de etiquetas. Es un parámetro más del modelo y se aprende con el resto.

Puntuar es recorrer y sumar

Se camina la secuencia sumando, en cada paso, lo que la red dice del token y lo que la tabla dice de la transición.

Las imposibles se hunden solas

Nadie escribe las reglas del esquema BIO. Las transiciones que no ocurren en los datos terminan con valores muy negativos.

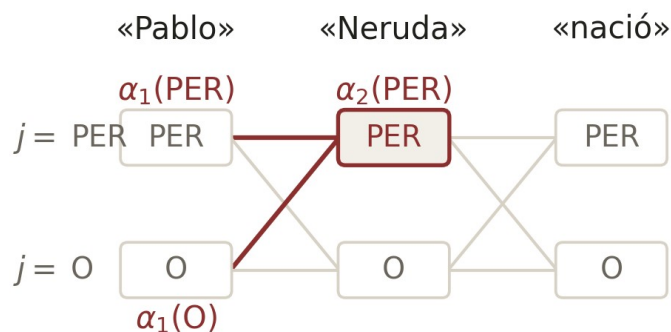
De dónde sale $Z(x)$

Para que los puntajes sean probabilidades hay que dividir por la suma de todos. Esa suma es Z .

$$p(y \mid x) = \frac{e^{s(x,y)}}{Z(x)} \quad \text{softmax sobre secuencias}$$

$$Z(x) = \sum_{y'} e^{s(x,y')} = \sum_{k=1}^K \alpha_n(k) \quad \text{la última columna}$$

$$\alpha_i(k) \equiv \sum_{y_{1:i}: y_i=k} e^{s(x, y_{1:i})} = \sum_{j=1}^K \alpha_{i-1}(j) e^{A_{j,k} + P_{i,k}} \quad j=\text{la etiqueta en } i-1$$



Z es el normalizador

El puntaje es un número cualquiera. Para que sea probabilidad hay que dividir por la suma sobre todas.

Depende de la oración, no de la etiqueta

Z se calcula una vez por oración y es la misma para todas sus candidatas.

Alfa vive en los nodos

Alfa de i y k suma los caminos que llegan a la etiqueta k en la posición i . Son n por K números, no K elevado a n .

De los embeddings a las etiquetas

Un vector estático por token abajo, la capa lineal más el CRF es el cabezal.

